The modules in the Write & Read paths are same just they're connected to different ports of the same module.

1. Write Path

This section covers the AW, W, and B channels. The Address (AW) and Data (W) paths are completely decoupled. Because AXI4 does not have a WID signal, we use a Token FIFO to guarantee that the W data beats perfectly track and follow the exact same physical path as their corresponding AW address.

1.1 Pipelined Skid Buffers

We must cut the combinatorial paths between our Slave logic and the external Master to hit high clock speeds. So, both the AW BUS and W BUS feed directly into Pipelined Skid Buffers. The AWREADY and WREADY signals sent back to the Master are generated only from the internal state of these Skid Buffers.

1.2 De-MUX & Token Synchronization

The incoming stream is split into parallel, per-ID queues (IDx AW FIFO and IDx W FIFO).

- **Address De-MUX:** Routes the AW payload into a specific queue based on the AWID.
- **Data De-MUX:** Routes the W payload into the matching data queue.
- **The Token Handshake:** When the AW De-MUX accepts an address, it pushes an ID token into the ID Token FIFO. The W De-MUX reads this token to know where to send the data. It only pops the token when it sees the WLAST bit go high. This token passing ensures the W beats strictly follow their AW address path.

1.3 The Cross-Group Per-ID Hazard Scoreboard

This block is the primary control structure preventing structural hazards (executing the same ID out-of-order across different latency paths). It guarantees strict same-ID ordering without the massive silicon area and power overhead of a Content Addressable Memory (CAM).

1.3.1 Internal Structure

Instead of searching through the internal FIFOs to see what is currently in-flight, the architecture tracks the pipeline state using two parallel arrays of Up/Down counters. There is one array dedicated to Group 0 and one dedicated to Group 1. Assuming a standard 4-bit AWID (16 unique IDs), each array contains sixteen 3-bit counters.

1.3.2 Stall & Release Mechanism

To guarantee strict in-order completion and valid BRESP sequencing, **the architecture strictly forbids transactions with the same ID from executing in different groups simultaneously**. The stall and release logic is governed by a purely combinatorial criss-cross check.

- **The Check:** When a transaction at the head of an IDx AW FIFO targets Group 0, the logic peeks at Group 1's active counter for that specific ID (and vice versa).
- **The Stall:** If the opposing group's counter is > 0, hazard_stall is asserted and the transaction is held in the FIFO. If the counter == 0, the transaction is clear to proceed through the Round Robin Arbiter.
- **State Updates (++ / --):**
    - **Increment (counter++):** Occurs the exact clock cycle a transaction successfully passes the arbiter and dispatches into its target group.
    - **Decrement (counter--):** Occurs the exact clock cycle a transaction finishes execution and pops from the In-Flight ID FIFO. We will decrement the specific counter depending on the popped ID & Group.
- **Auto-Release:** Because the stall wire is combinational, no state machine is required to release it. The stall instantly drops the exact cycle the opposing group's counter decrements to 0.

1.4 Group Arbitration & The Arbiter Token Handshake

Because the frontend splits transactions into parallel, per-ID queues, multiple unstalled IDs may target the same Execution Group simultaneously. The architecture uses a dedicated Round Robin Arbiter and a secondary token-passing mechanism to merge these queues while keeping the decoupled AW and W channels perfectly synchronized.

- **The AW Arbiter:** Each Group has a dedicated Round Robin Arbiter on the Address path. It monitors all unstalled IDx AW FIFOs. When it grants access to a winning ID, it pushes that address into the Group's ADDR FIFO.
- **The Token Push:** The exact clock cycle the AW Arbiter grants a request, it pushes an "Arbiter Token" (the ID of the winning queue) into the **Arbiter Token FIFO**.
- **The W Channel Data MUX:** On the Data path, a multiplexer connects all the IDx W FIFOs to the Group's internal Data Steering Unit. The select line for this MUX is driven directly by the token sitting at the head of the Arbiter Token FIFO.
- **The Token Pop (Burst Routing):** Because AXI4 data arrives in bursts, the W MUX must hold its selection for multiple clock cycles. The MUX continuously routes data from the winning ID queue and only pops the Arbiter Token FIFO when it sees the WLAST bit go high. This guarantees the entire data payload finishes routing before the MUX switches to the next winning ID.

1.5 Group Dispatch & Data Steering

- **Address Gen Unit:** Unrolls the AXI burst into individual beat addresses. It pushes a token into the Module Token FIFO. (It used Run Length Encoding & an ADDR Decoder FSM for reducing the depth of the Control FIFO.)
- **Data Steering Unit:** Pops the Module Token FIFO to route the incoming W data beats to the correct module.

1.6 Fixed Execution Latencies & Completion Tracking

Once data crosses into the modules, execution is completely deterministic. The data & control signals must be held till completion.

- **In-Flight Tracking (The Release):** Because WLAST is held stable at the FIFO output throughout the execution, no internal tracking state machines are required. The logical AND of the module's complete pulse and the stable WLAST bit generates a 1-cycle pop_pulse.
- **The OR Gate:** Because dispatch is strictly in-order, it is physically impossible for two modules within the same group to fire a pop_pulse simultaneously. The pop_pulse wires from all modules are simply ORed together to safely pop the In-Flight ID FIFO and decrement the frontend hazard counters.

1.7 Backend Response (B-Channel) & AXI Compliance

The backend merges the completed transactions from both groups and safely manages the physical out-of-order interleaving without violating the AXI4 specification.

- **B-Channel Arbitration:** The popped IDs from Group 0 and Group 1 arrive at a backend Round Robin Arbiter. The winning response is pushed into a B FIFO, followed by a Pipelined Skid Buffer, before finally hitting the B BUS.
- **AXI Compliance:** The AXI4 specification dictates that transactions with the same ID must remain strictly in order, but transactions with different IDs can interleave freely. Because the frontend Cross-Group Hazard Scoreboard physically prevents the same ID from entering different groups simultaneously, the responses arriving at the backend arbiter are mathematically safe. Different IDs naturally interleave to maximize throughput, while same-ID sequences remain locked in their single-group FIFO, guaranteeing 100% AXI ordering compliance.

2. Read Path

This section covers the AR (Address Read) and R (Read Data) channels. The Read path is naturally more streamlined than the Write path because it only requires routing the incoming address and returning the requested data, without a parallel incoming data stream to synchronize.

2.1 Pipelined Skid Buffers

'SAME AS WRITE PATH'

2.2 De-MUX & ID Splitting

The incoming AR stream is combinationally split into parallel, per-ID queues (IDx AR FIFO).

- **Address De-MUX:** Routes the AR payload into a specific queue based on the ARID.
- Unlike the Write path, no Token FIFO is needed at the frontend because there is no incoming W channel to synchronize.

2.3 The Cross-Group Per-ID Hazard Scoreboard

'SAME AS WRITE PATH'

2.4 Group Arbitration

'SAME AS WRITE PATH, WITHOUT THE DATA'

2.5 Group Dispatch & Execution Latencies

- **Address Gen Unit (AGU):** Unrolls the AXI read burst. Unlike the Write path, the Read AGU dynamically generates a last bit flag for the final beat of the burst. The control tuple {address, run_length, last} is pushed into the target module's Control FIFO.
- **Execution:** ADDR Decoder FSMs process the control tuple, fetch the requested memory data, and push the raw data to their internal RESP FIFO.

2.6 Completion Tracking & Payload Assembly

Because the Read channel returns multiple beats per burst, the pipeline dynamically manages the RLAST signal and the ID tracking without complex state machines.

- **RLAST Generation:** The RLAST signal is dynamically generated by executing a logical AND on the module's complete pulse and the AGU-provided last bit.
- **RID Peek vs. Pop:** The pipeline must attach the correct RID to every outgoing beat. The logic reads the In-Flight ID FIFO. For intermediate beats (RLAST == 0), the ID is safely **peeked**. On the final beat (RLAST == 1), the ID is **popped**, which simultaneously fires the pulse to decrement the frontend hazard counters and release the lock.
- **Module MUXing:** Because dispatch is strictly in-order, it is physically impossible for two modules within the same group to complete a beat simultaneously. The encoded module_complete pulses act as select lines to MUX the module outputs to the Group Arbiter.

2.7 Backend Response (R-Channel) & AXI Compliance

The backend merges the retrieved data from both groups, assembles the final AXI payload, and safely manages physical out-of-order interleaving.

- **Payload Concatenation:** Every read beat exiting the group is concatenated into the final AXI4 format: {RID, RDATA, RLAST}.

- **R-Channel Arbitration:** The fully assembled read beats from Group 0 and Group 1 arrive at a backend Round Robin Arbiter. The winning response beat is pushed into a final RESP FIFO, followed by a Pipelined Skid Buffer, before finally driving the R BUS.
- **AXI Compliance:** The AXI4 specification permits read data from different IDs to interleave. Because the frontend Cross-Group Hazard Scoreboard physically prevents the same ID from entering different groups simultaneously, the response beats arriving at the backend arbiter naturally and safely interleave different IDs to maximize throughput. Same-ID sequences remain locked to a single group's pipeline, guaranteeing 100% in-order AXI compliance.

END OF DOC.